



PABNA UNIVERSITY OF SCIENCE AND TECHNOLOGY

LAB REPORT

Course Code: ICE-3202

Course Title: Network Programming with JAVA Sessional

Submitted By:

Apurbo Sharma

Roll : 220604

Student of 2021-22 session

Department of ICE, PUST

Submitted To :

Taskin Noor Turna

Assistant Professor

Department of ICE, PUST

Faculty of Engineering & Technology
Department of Information and Communication Engineering

Submission Date :

signature:

B.Sc.(Engineering) 3rd Year 2nd Semester Examination-2024
Session: 2021-22 Course Code: ICE-3202
Course Title: Network Programming with Java Sessional

Problem List

1.	Write a JAVA Program to Display Image using JFrame
2.	Write a JAVA Program for generating Restaurant Bill
3.	Write a JAVA Program to Create a Student form in GUI
4.	Write a JAVA Program to develop a simple calculator in GUI
5.	Write a Java program to illustrate suspend, resume and stop operation of thread.
6.	Write a Java program to create thread class.
7.	Write a Java program to illustrate yield(), stop() and sleep() method using thread.
8.	Write a Java program to use priority of thread.
9.	Write a client-server TCP socket program in java that the server listens for connection requests, and whatever message the client sends, the server converts it to uppercase and sends it back.
10.	Write a client-server UDP socket program in java that the server listens for connection requests, and whatever message within 1024 bytes the client sends, the server converts it to uppercase and sends it back after 6 ms.

Problem No - 01: Write a JAVA Program to Display Image using JFrame

Source Code :

```
import java.awt.FlowLayout;

import javax.swing.ImageIcon; import
javax.swing.JFrame; import javax.swing.JLabel;

public class Image extends JFrame { private
    ImageIcon image1; private JLabel label1;
    private ImageIcon image2; private JLabel
    label2;

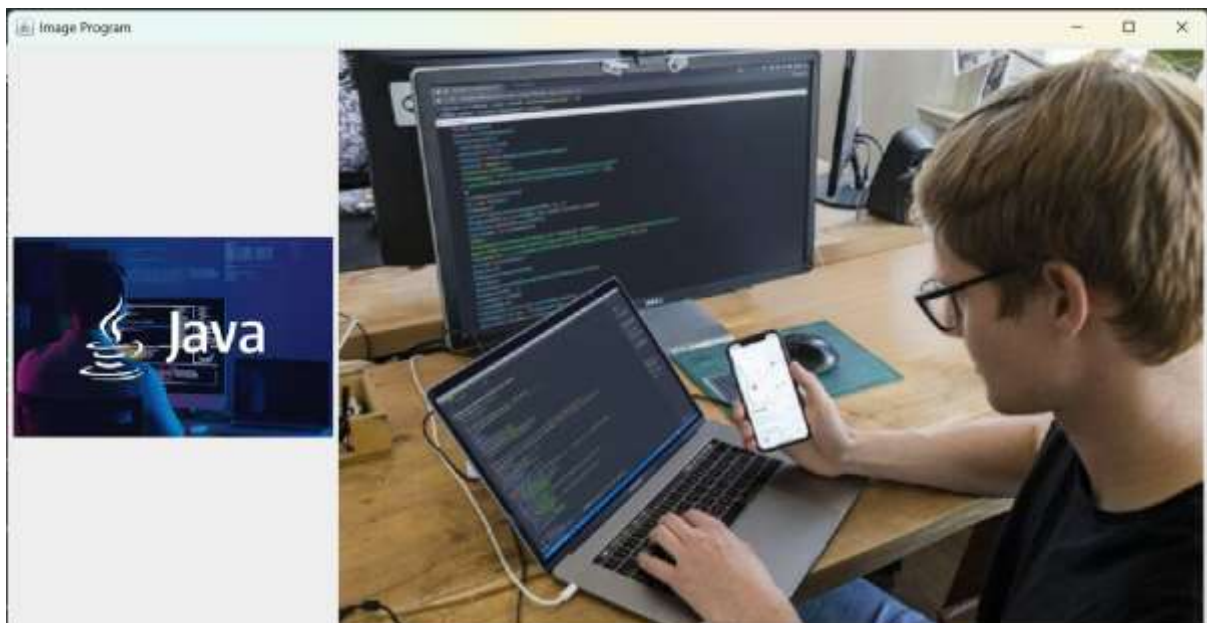
    Image(){ setLayout(new FlowLayout());

        image1 = new ImageIcon(getClass().getResource("java_logo.jpg")); label1 = new
        JLabel(image1); add(label1);

        image2 = new ImageIcon(getClass().getResource("man_with_pc.png")); label2 = new
        JLabel(image2); add(label2);
    }

    public static void main(String args[]) { Image gui = new Image();
        gui.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); gui.setVisible(true);
        gui.pack(); gui.setTitle("Image Program");
    }
}
```

Output:



Problem No - 02: Write a JAVA Program for generating Restaurant Bill.

Source Code :

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class BillGeneration extends JFrame implements ActionListener {

    JLabel l;
    JCheckBox cb1, cb2, cb3;
    JButton b;

    BillGeneration() {
        l = new JLabel("Cafeteria Billing System");
        l.setBounds(50, 50, 300, 20);

        cb1 = new JCheckBox("Sandwich @ 80");
        cb1.setBounds(100, 100, 150, 20);

        cb2 = new JCheckBox("Coffee @ 50");
        cb2.setBounds(100, 150, 150, 20);

        cb3 = new JCheckBox("Juice @ 60");
        cb3.setBounds(100, 200, 150, 20);

        // ✓ Updated Button
        b = new JButton("Place Order");
        b.setBounds(100, 250, 150, 40);
        b.setFocusable(false);
        b.setBackground(Color.BLACK);
        b.setForeground(Color.WHITE);
        b.addActionListener(this);

        add(l); add(cb1); add(cb2); add(cb3); add(b);

        setSize(400, 400);
        setLayout(null);
        setVisible(true);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }

    public void actionPerformed(ActionEvent e) {
        float amount = 0;
        String msg = "";

        if (cb1.isSelected()) {
            amount += 80;
        }
    }
}
```

```

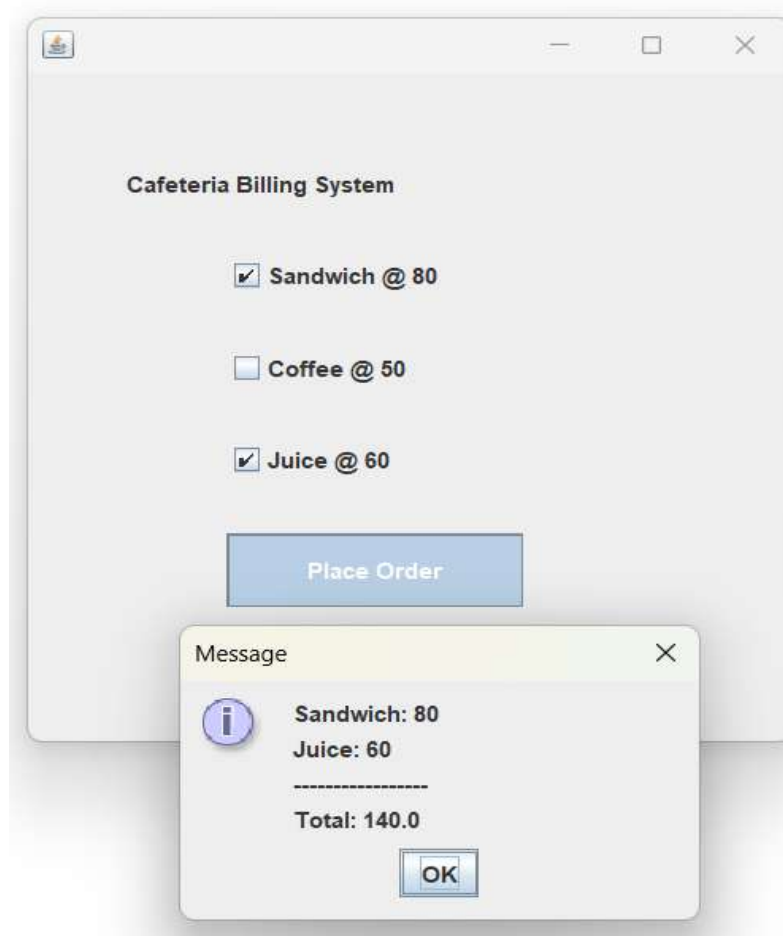
        msg = "Sandwich: 80\n";
    }
    if (cb2.isSelected()) {
        amount += 50;
        msg += "Coffee: 50\n";
    }

    if (cb3.isSelected()) {
        amount += 60;
        msg += "Juice: 60\n";
    }
    msg += "-----\n";
    JOptionPane.showMessageDialog(this, msg + "Total: " + amount);
}

public static void main(String[] args) {
    new BillGeneration();
}
}

```

Output:



Problem No - 03: Write a JAVA Program to Create a Student form in GUI.

Source Code :

```
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.*;

public class form implements ActionListener {

    private static JLabel success;
    private static JFrame frame;
    private static JLabel label1, label2, label3, title;
    private static JPanel panel;
    private static JButton button;
    private static JTextField userText1, userText2, userText3;

    public static void main(String[] args) {

        frame = new JFrame("Student Form");
        panel = new JPanel();
        frame.setSize(420, 320);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.add(panel);
        panel.setLayout(null);
        // Title
        title = new JLabel("Student Information Form");
        title.setBounds(90, 10, 250, 25);
        title.setFont(new Font("Arial", Font.BOLD, 14));
        panel.add(title);
        // Labels
        label1 = new JLabel("Name:");
        label1.setBounds(30, 50, 80, 25);
        panel.add(label1);

        label2 = new JLabel("Roll:");
        label2.setBounds(30, 90, 80, 25);
        panel.add(label2);

        label3 = new JLabel("Department:");
        label3.setBounds(30, 130, 100, 25);
        panel.add(label3);
        // TextFields
        userText1 = new JTextField();
        userText1.setBounds(140, 50, 200, 25);
        panel.add(userText1);
        userText2 = new JTextField();
        userText2.setBounds(140, 90, 200, 25);
```

```

panel.add(userText2);
userText3 = new JTextField();
userText3.setBounds(140, 130, 200, 25);
panel.add(userText3);
// Button (styled)
button = new JButton("Submit");
button.setBounds(150, 180, 120, 35);
button.setBackground(Color.DARK_GRAY);
button.setForeground(Color.WHITE);
button.setFocusable(false);
button.addActionListener(new form());
panel.add(button);
// Success label
success = new JLabel("");
success.setBounds(100, 230, 300, 25);
success.setForeground(Color.BLUE);
panel.add(success);
frame.setVisible(true);
}
@Override
public void actionPerformed(ActionEvent e) {
    String name = userText1.getText();
    String roll = userText2.getText();
    String dept = userText3.getText();

    success.setText("Saved: " + name + ", " + roll + ", " + dept);
}
}

```

Output:

The screenshot shows a Java Swing window titled "Student Form". Inside the window, there is a section titled "Student Information Form". This section contains three labeled text input fields: "Name:" with the value "Apurbo Sharma", "Roll:" with the value "220604", and "Department:" with the value "ICE". Below these fields is a dark gray button labeled "Submit". At the bottom of the window, a blue text label displays the message "Saved: Apurbo Sharma, 220604, ICE", indicating that the form data has been successfully saved.

Problem No - 04: Write a JAVA Program to develop a simple calculator in GUI.

Source Code :

```
import java.awt.*;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;

import javax.swing.*;

public class calculator extends JFrame implements ActionListener {
    JButton b10, b11, b12, b13, b14, b15; JButton b[] = new
    JButton[10]; int i, r, n1, n2; JTextField res; char op; public
    calculator() { super("Calulator"); setLayout(new
    BorderLayout());
        JPanel p = new JPanel();
        p.setLayout(new GridLayout(4, 4)); for (int i = 0; i <= 9;
        i++) {
            b[i] = new JButton(i + ""); p.add(b[i]);
            b[i].addActionListener(this);
        }
        b10 = new JButton("+"); p.add(b10);
        b10.addActionListener(this); b11 = new
        JButton("-"); p.add(b11);
        b11.addActionListener(this); b12 = new
        JButton("*"); p.add(b12);
        b12.addActionListener(this); b13 = new
        JButton("/"); p.add(b13);
        b13.addActionListener(this); b14 = new
        JButton("="); p.add(b14);
        b14.addActionListener(this); b15 = new
        JButton("C"); p.add(b15);
        b15.addActionListener(this); res = new
        JTextField(10); add(p,
        BorderLayout.CENTER); add(res,
        BorderLayout.NORTH); setVisible(true);
        setSize(200, 200);
    }

    public void actionPerformed(ActionEvent ae) { JButton pb = (JButton)
    ae.getSource(); if (pb == b15) { r = n1 = n2 = 0; res.setText("");
    }
    else if (pb == b14) {
        n2 = Integer.parseInt(res.getText()); eval(); res.setText("")
        + r);
    }
    else {
        boolean opf = false; if (pb == b10) { op = '+'; opf = true; } if
        (pb == b11) { op = '-'; opf = true; } if (pb == b12) { op = '*'; opf
        = true; } if (pb == b13) { op = '/'; opf = true; } if (!opf) {
            for (i = 0; i < 10; i++) { if (pb == b[i]) {
                String t = res.getText(); t += i;
                res.setText(t);
            }
        }
    }
}
```

```

    }
}
else {
    n1 = Integer.parseInt(res.getText()); res.setText("");
}
}
}
int eval() {
    switch (op) {
        case '+': r = n1 + n2; break; case '-': r = n1 - n2;
        break; case '*': r = n1 * n2; break; case '/': r = n1 /
        n2; break;
    }
    return r;
}
public static void main(String arg[]) {
    new calculator();
}
}

```

Output:



Problem No - 05: Write a Java program to illustrate suspend, resume and stop operation of thread.

Source Code :

```
class sus_res_stop implements Runnable {
    Thread th;
    boolean suspend_flag, stop_flag;

    sus_res_stop(String name) {
        th = new Thread(this, name);
        suspend_flag = false;
        stop_flag = false;
        th.start();
    }

    public void run() {
        try {
            int j = 1;
            while (j++ < 20) {
                synchronized (this) {
                    while (suspend_flag)
                        wait();

                    if (stop_flag)
                        break;
                }
            }
        } catch (InterruptedException e) {
            System.out.println("Thread interrupted");
        }
    }

    synchronized void my_suspend() {
        suspend_flag = true;
    }

    synchronized void my_resume() {
        suspend_flag = false;
        notify();
    }

    synchronized void my_stop() {
        suspend_flag = false;
        stop_flag = true;
        notify();
    }
}

public class eg_SRS {
    public static void main(String args[]) {
        try {
            sus_res_stop S_R_S_T = new sus_res_stop("SRS");

            System.out.println("Thread S_R_S_T is created and started");
            ;
            Thread.sleep(2000);
        }
    }
}
```

```

        S_R_S_T.my_suspend();
        System.out.println("Thread S_R_S_T is suspended");
        Thread.sleep(2000);

        S_R_S_T.my_resume();
        System.out.println("Thread S_R_S_T is resumed");
        Thread.sleep(2000);

        S_R_S_T.my_stop();
        System.out.println("Thread S_R_S_T is stopped");
    } catch (InterruptedException e) {
        System.out.println("Generated interrupted exception");
    }
}

```

Output:

```

Thread S_R_S_T is created and started
Thread S_R_S_T is suspended
Thread S_R_S_T is resumed
Thread S_R_S_T is stopped

```

Problem No - 06: Write a Java program to create thread class.

Source Code :

```
class A extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("From Thread A : i = " + i);
        }
        System.out.println("Exit from A");
    }
}

class B extends Thread {
    public void run() {
        for (int j = 1; j <= 5; j++) {
            System.out.println("From Thread B : j = " + j);
        }
        System.out.println("Exit from B");
    }
}

class C extends Thread {
    public void run() {
        for (int k = 1; k <= 5; k++) {
            System.out.println("From Thread C : k = " + k);
        }
        System.out.println("Exit from C");
    }
}

class ThreadTest {
    public static void main(String args[]) {
        new A().start();
        new B().start();
        new C().start();
    }
}
```

Output:

```
From Thread A : i = 1
From Thread A : i = 2
From Thread A : i = 3
From Thread A : i = 4
From Thread A : i = 5
Exit from A
```

```
From Thread C : k = 1
From Thread C : k = 2
From Thread C : k = 3
From Thread C : k = 4
From Thread C : k = 5
Exit from C
```

```
From Thread B : j = 1
From Thread B : j = 2
From Thread B : j = 3
From Thread B : j = 4
From Thread B : j = 5
Exit from B
```

Problem No - 07: Write a Java program to illustrate yield(), stop() and sleep() method using thread.

Source Code :

```
class A extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            if (i == 1)
                Thread.yield();

            System.out.println("From Thread A : i = " + i);
        }
        System.out.println("Exit from A");
    }
}

class B extends Thread {
    public void run() {
        for (int j = 1; j <= 5; j++) {
            System.out.println("From Thread B : j = " + j);

            if (j == 3)
                break;
        }
    }
}

class C extends Thread {
    public void run() {
        for (int k = 1; k <= 5; k++) {
            System.out.println("From Thread C : k = " + k);

            if (k == 1) {
                try {
                    Thread.sleep(1000);
                } catch (Exception e) {
                }
            }
        }
        System.out.println("Exit from C");
    }
}

class ThreadMethods {
    public static void main(String args[]) {
        A threadA = new A();
        B threadB = new B();
        C threadC = new C();

        System.out.println("Start thread A");
        threadA.start();

        System.out.println("Start thread B");
        threadB.start();

        System.out.println("Start thread C");
        threadC.start();
    }
}
```

```
        System.out.println("End of main thread");  
    }  
}
```

Output:

```
Start thread A  
Start thread B  
Start thread C  
End of main thread
```

```
From Thread A : i = 1  
From Thread B : j = 1  
From Thread C : k = 1
```

```
From Thread B : j = 2  
From Thread B : j = 3
```

```
From Thread A : i = 2  
From Thread A : i = 3  
From Thread A : i = 4  
From Thread A : i = 5  
Exit from A
```

```
From Thread C : k = 2  
From Thread C : k = 3  
From Thread C : k = 4  
From Thread C : k = 5  
Exit from C
```

Problem No - 08: Write a Java program to use priority of thread.

Source Code :

```
class A extends Thread {
    public void run() {
        System.out.println("threadA started");
        for (int i = 1; i <= 4; i++) {
            System.out.println("From Thread A : i = " + i);
        }
        System.out.println("Exit from A");
    }
}

class B extends Thread {
    public void run() {
        System.out.println("threadB started");
        for (int j = 1; j <= 4; j++) {
            System.out.println("From Thread B : j = " + j);
        }
        System.out.println("Exit from B");
    }
}

class C extends Thread {
    public void run() {
        System.out.println("threadC started");
        for (int k = 1; k <= 4; k++) {
            System.out.println("From Thread C : k = " + k);
        }
        System.out.println("Exit from C");
    }
}

class ThreadPriority {
    public static void main(String[] args) {
        A threadA = new A();
        B threadB = new B();
        C threadC = new C();

        threadC.setPriority(Thread.MAX_PRIORITY);
        threadB.setPriority(threadA.getPriority() + 1);
        threadA.setPriority(Thread.MIN_PRIORITY);

        System.out.println("Start thread A");
        threadA.start();

        System.out.println("Start thread B");
        threadB.start();

        System.out.println("Start thread C");
        threadC.start();

        System.out.println("End of main thread");
    }
}
```

Output:

```
Start thread A
Start thread B
Start thread C
End of main thread

threadB started
From Thread B : j = 1
From Thread B : j = 2

threadA started
From Thread A : i = 1
From Thread A : i = 2
From Thread A : i = 3
From Thread A : i = 4
Exit from A

threadC started
From Thread C : k = 1

From Thread B : j = 3
From Thread B : j = 4
Exit from B

From Thread C : k = 2
From Thread C : k = 3
From Thread C : k = 4
Exit from C
```

Problem No - 09: Write a client-server TCP socket program in java that the server listens for connection requests, and whatever message the client sends, the server converts it to uppercase and sends it back.

Source Code (Server):

```
import java.io.*;
import java.net.*;

public class TCPServer {
    public static void main(String[] args) throws Exception {
        ServerSocket ss = new ServerSocket(5000);
        System.out.println("Server is waiting...");
        Socket s = ss.accept();
        System.out.println("Client connected");
        BufferedReader br = new BufferedReader(
            new InputStreamReader(s.getInputStream()));
        PrintWriter pw = new PrintWriter(s.getOutputStream(), true);
        String msg = br.readLine();
        System.out.println("Received: " + msg);
        pw.println(msg.toUpperCase());
        s.close();
        ss.close();
    }
}
```

Source Code (Client):

```
import java.io.*;
import java.net.*;

public class TCPClient {
    public static void main(String[] args) throws Exception {
        Socket s = new Socket("localhost", 5000);
        BufferedReader input = new BufferedReader(
            new InputStreamReader(System.in));
        PrintWriter pw = new PrintWriter(s.getOutputStream(), true);
        BufferedReader br = new BufferedReader(
            new InputStreamReader(s.getInputStream()));
        System.out.print("Enter message: ");
        String msg = input.readLine();
        pw.println(msg);

        String response = br.readLine();
        System.out.println("Server response: " + response);

        s.close();
    }
}
```

Output:

```
Server is waiting...
Client connected
Received: hello

Enter message: hello
Server response: HELLO
```

Problem No - 10: Write a client-server UDP socket program in java that the server listens for connection requests, and whatever message within 1024 bytes the client sends, the server converts it to uppercase and sends it back after 6 ms.

Source Code (Server):

```
import java.net.*;

public class UDPServer {
    public static void main(String[] args) throws Exception {
        DatagramSocket ds = new DatagramSocket(5000);
        byte[] buffer = new byte[1024];

        System.out.println("UDP Server waiting...");

        DatagramPacket dp = new DatagramPacket(buffer, buffer.length);
        ds.receive(dp);

        String msg = new String(dp.getData(), 0, dp.getLength());
        System.out.println("Received: " + msg);

        Thread.sleep(6);

        String upper = msg.toUpperCase();
        byte[] sendData = upper.getBytes();

        DatagramPacket sendPacket = new DatagramPacket(
            sendData, sendData.length,
            dp.getAddress(), dp.getPort());

        ds.send(sendPacket);
        ds.close();
    }
}
```

Source Code (Client):

```
import java.net.*;
import java.io.*;

public class UDPClient {
    public static void main(String[] args) throws Exception {
        DatagramSocket ds = new DatagramSocket();

        BufferedReader input = new BufferedReader(
            new InputStreamReader(System.in));

        System.out.print("Enter message: ");
        String msg = input.readLine();

        byte[] sendData = msg.getBytes();
        InetAddress ip = InetAddress.getByName("localhost");

        DatagramPacket dp = new DatagramPacket(
            sendData, sendData.length, ip, 5000);

        ds.send(dp);
    }
}
```

```
byte[] buffer = new byte[1024];
DatagramPacket receivePacket = new DatagramPacket(buffer,
    buffer.length);
ds.receive(receivePacket);

String response = new String(
    receivePacket.getData(), 0, receivePacket.getLength());

System.out.println("Server response: " + response);

ds.close();
    }
}
```

Output:

```
UDP Server waiting...
Received: java
```

```
Enter message: java
Server response: JAVA
```